Vite & Gourmand

Gestion de projet

Organisation, découpage du travail, suivi et gestion des versions

ECF – Titre professionnel Développeur Web et Web Mobile (Studi)

Auteur : Quentin Cire

Site : https://vite-gourmand-kappa.vercel.app

Dépôt : https://github.com/Bossim-Rack/V-G

Version du 29 octobre 2026

1. Organisation générale

Le projet a été mené seul, dans le rôle du développeur de l'agence FastDev, à partir des besoins recueillis par le chef de projet (le sujet de l'ECF). L'objectif était de livrer une application **en ligne et fonctionnelle**, l'absence de mise en ligne entraînant des pénalités.

Élément	Choix
Outil de gestion de projet	à compléter : Trello, Notion, Jira…
Lien vers le tableau	à compléter
Méthode	Tableau Kanban : une carte par fonctionnalité, qui passe de « À faire » à « En cours », « À tester » puis « Terminé » (à confirmer ou adapter)
Gestion des versions	Git et GitHub, avec une branche par fonctionnalité (voir partie 4)
Documentation	Dossier `docs/` du dépôt et `README.md`

> À compléter : une capture du tableau de gestion de projet, avec ses colonnes et quelques cartes.

2. Découpage du travail

Le sujet a été découpé en **fonctionnalités indépendantes**, regroupées par espace de l'application. Chacune est devenue une carte du tableau, puis une branche Git une fois le travail commencé.

Lot	Fonctionnalités	État
Socle	Routeur du front, en-tête et pied de page communs, API Symfony, base MySQL, authentification par jeton	Terminé
Visiteur	Accueil (présentation, savoir-faire, avis validés), horaires dans le pied de page, mentions légales et CGV	Terminé
Menus	Vue globale, filtres dynamiques (prix maximum, fourchette, thème, régime, allergène, nombre de personnes), fiche détaillée avec les conditions	Terminé
Comptes	Inscription (mot de passe sécurisé), mail de bienvenue avec confirmation, connexion, mot de passe oublié, modification du profil, suppression du compte	Terminé
Commande	Formulaire pré-rempli, calcul du prix (réduction de 10 %, livraison au kilomètre), conditions à accepter, mail de confirmation, stock	Terminé
Espace client	Liste des commandes, modification et annulation tant que la commande est en attente, suivi daté, avis (note et commentaire)	Terminé
Espace employé	Gestion des menus, plats, thèmes, régimes et horaires ; filtre des commandes ; changements de statut avec mails automatiques ; modification ou annulation après contact ; modération des avis	Terminé
Espace administrateur	Création et désactivation des comptes employés, statistiques MongoDB (graphique, chiffre d'affaires filtrable)	Terminé
Contact	Formulaire envoyé par mail à l'entreprise	Terminé
Mise en ligne	Docker, Render, Aiven, MongoDB Atlas, Brevo, Vercel	Terminé
Qualité	Accessibilité (RGAA), tests PHPUnit, nettoyage du code	Terminé
Livrables	README, fichiers SQL, manuel d'utilisation, charte graphique, documentation technique, documentation de gestion de projet	Terminé

Pour chaque fonctionnalité, la carte n'est passée à « Terminé » qu'après un test du parcours complet dans le navigateur (en local, puis sur le site en ligne).

3. Étapes du projet

À vérifier : ajuster l'ordre et le contenu des étapes à ton déroulement réel.

1. **Analyse du sujet** : liste des besoins par profil (visiteur, client, employé, administrateur) et identification des règles de gestion (réduction, livraison, statuts, avis).
2. **Modélisation** : modèle conceptuel de données réalisé avec draw.io (dossier `Suivie Projet/` du dépôt), puis traduit en entités Doctrine.
3. **Maquettage** : wireframes puis maquettes des pages principales, en version bureau et mobile, avec la palette et les polices de la charte graphique.
4. **Intégration du front** : pages HTML et CSS avec Bootstrap, puis le routeur JavaScript.
5. **Développement de l'API** : entités, contrôleurs et règles métier, page par page, en branchant le front sur l'API au fur et à mesure.
6. **Sécurité et mails automatiques** : confirmation de l'email, liens de réinitialisation, droits par rôle, suivi et annulation des commandes, statistiques MongoDB, suppression de compte.
7. **Déploiement** : préparation de l'image Docker, puis création des services en ligne.
8. **Recette en ligne** : test de chaque parcours sur le site déployé et corrections (mails, fuseau horaire, allergènes, affichage).
9. **Finalisation** : accessibilité, nettoyage du code et rédaction des livrables.

Les dernières étapes sont visibles dans l'historique Git :

Date	Étape (branche ou commit)
23 juillet 2026	Version du projet déposée sur GitHub, premier README, sauvegarde SQL
27 octobre 2026	`feature/mails-securite-commandes` : mails automatiques, sécurité de l'API, suivi et annulation des commandes, statistiques MongoDB, suppression de compte
27 octobre 2026	`feature/deploiement` : Docker, initialisation de la base, mails via l'API Brevo, configuration Vercel, guide de déploiement
28 octobre 2026	`feature/dockerfile-racine`, `feature/config-production` : ajustements pour Render et URL de l'API en ligne
28 octobre 2026	`feature/allergenes-affichage`, `feature/fuseau-horaire` : corrections issues de la recette en ligne
29 octobre 2026	`feature/textes-site`, `feature/livrables` : textes définitifs, accessibilité, nettoyage, fichiers SQL et documentation

À compléter : les dates de début du projet et des étapes 1 à 5, réalisées avant le premier dépôt sur GitHub.

4. Gestion des versions (Git)

Le dépôt GitHub public (https://github.com/Bossim-Rack/V-G) applique les bonnes pratiques demandées :

- `main` : branche principale, en production. Chaque push sur `main` déclenche le redéploiement automatique du front (Vercel) et de l'API (Render).
- `develop` : branche de développement, qui réunit les fonctionnalités terminées.
- `feature/<nom>` : une branche par fonctionnalité, créée depuis `develop`.

Cycle d'une fonctionnalité :

```
git checkout develop
git checkout -b feature/nom-de-la-fonctionnalite
#   ... développement et commits ...
#   ... test de la fonctionnalité en local ...
git checkout develop
git merge --no-ff feature/nom-de-la-fonctionnalite
#   ... test de develop ...
git checkout main
git merge develop
git push origin main develop feature/nom-de-la-fonctionnalite
```

- **Messages de commit** : rédigés en français et décrivant le changement (« Fuseau horaire Europe/Paris pour PHP », « Allergènes créés à l'initialisation de la base »...).
- **Secrets** : jamais versionnés. Le fichier `.env.local` est ignoré par Git, et les clés de production n'existent que dans les variables d'environnement de Render.

5. Suivi de la qualité

- **Recette** : chaque parcours du sujet est testé à la main (inscription, commande, statuts, avis, gestion du catalogue, administration), en local puis en ligne.
- **Tests automatisés** : PHPUnit sur la connexion, les droits et l'entité `User`.
- **Accessibilité** : audit automatique (axe-core, règles WCAG 2.1 A et AA) sur toutes les pages, en version bureau et mobile, puis correction jusqu'à zéro erreur.
- **Base de données** : `doctrine:schema:validate` vérifie que les fichiers SQL et les entités décrivent la même base.

6. Bilan

Toutes les fonctionnalités du sujet sont réalisées et en ligne. Les principales difficultés ont été rencontrées lors de la mise en ligne :

- l'envoi de mails, bloqué en SMTP par l'hébergeur et remplacé par l'API HTTP de Brevo ;
- la configuration des différents services gratuits et de leurs variables ;
- les différences entre l'environnement local et le serveur Linux : sensibilité à la casse des chemins, fuseau horaire.

Pistes d'amélioration :

- limiter le nombre de tentatives de connexion ;
- proposer une galerie de plusieurs photos par menu ;
- passer l'API sur une offre d'hébergement qui ne se met pas en veille.